מספר התלמיד הנבחן
רשום את כל תשע הספרות

הדבק כאן את מדבקת הנבחן

האוניברסיטה הפתוחה

כ"ד בסיון תשע"ט

27
ביוני 2019

סמסטר 2019ב

4 / 20905

מס' שאלון - 517

מס' מועד 83

שאלון בחינת גמר

20905 - שפות תכנות

משך בחינה: 3 שעות

בשאלון זה **9** עמודים

מבנה הבחינה:

בבחינה ארבע שאלות.

עליכם לענות על כל השאלות.

משקל השאלות מפורט בגוף השאלות.

שימו לב:

את תשובותיכם רשמו אך ורק במחברת הבחינה.

כתבו בכתב יד ברור ומסודר.

חומר עזר:
ספר הקורס: essentials of programming languages
מדריך הלמידה. מותרות הערות בכתב יד, ע"ג הספרים.
אין להכניס חומר מודפס או כל חומר אחר מכל סוג שהוא.

בהצלחה !!!

אינכם חייבים
להחזיר את השאלון לאוניברסיטה הפתוחה

שאלה 1 (25 נקודות)

הרחיבו את שפת LET (שפת "ויהי") כך שביטוי if יבוצע על פי הדקדוק הבא:

להלן הדקדוק המגדיר את התחביר החדש עבור ביטוי if :

Expression* ::= if *Expression* then *Expression
else *Expression*

if-exp (bool exp1 exp2)

התחביר זהה לחלוטין לתחביר הקיים כבר בשפה, אלא שעליו להיות מבוצע בצורה שונה, כפי שיוסבר להלן:

- bool ביטוי בוליאני עבור ה-if. הביטוי יכול להיות הן בוליאני, והן חשבוני, אם מדובר בביטוי חשבוני אזי הוא ייחשב כ-true אם ערכו שונה מאפס, ו-false אם ערכו אפס. (זה השדרוג הרצוי ביחס ל-if הקיים כרגע בשפה שתומך רק במתן ביטוי בוליאני בלבד)
- exp1 ביטוי שערכו יוחזר כאשר bool הוא true
- exp2 ביטוי שערכו יוחזר כאשר bool הוא false

להלן דוגמא להמחשת השימוש בביטוי החדש:

```
let x= 7
in
  let z= if –(x,9) then 70 else 90
 in  -(z,x)
```

בדוגמה זו, התנאי הבוליאני של ה-if הוא true מאחר והביטוי (x,9)– שונה מאפס, ולכן, משתנה z יקבל את הערך 70. ולכן ערכה של הדוגמה הוא : (num-val 63)

א. (7 נקודות)

להלן נתונים ה-scanner, וה-parser של שפת "ויהי". כתבו מהו השינוי הדרוש בדקדוק של ביטוי if כך שיתאים להגדרת הדקדוק החדש. **<u>את תשובותיכם כתבו אך ורק במחברת הבחינה.</u>**

ה-scanner:

```
(define the-lexical-spec
   '((whitespace (whitespace) skip)
     (comment ("%" (arbno (not #\newline))) skip)
     (identifier (letter (arbno (or letter digit "_" "-" "?"))) symbol)
     (number (digit (arbno digit)) number)
     (number ("-" digit (arbno digit)) number)
   ))
```

(המשך השאלה בעמוד הבא)

ה-parser:

```
(define the-grammar
  '((program (expression) a-program)
   (expression (number) const-exp)
   (expression ("-" "(" expression "," expression ")") diff-exp)
   (expression ("zero?" "(" expression ")") zero?-exp)
   (expression ("if" expression "then" expression "else" expression) if-exp)
   (expression (identifier) var-exp)
   (expression ("let" identifier "=" expression "in" expression) let-exp)
  ))
```

ב. (18 נקודות)

להלן הפרוצדורה *value-of* של שפת **"ויהי"** כפי שמופיעה בספר הלימוד. יש לעדכנה כך שתחשב את ביטוי if על פי הדקדוק החדש. **<u>את התשובה כיתבו במחברת הבחינה</u>**.

הערות:

- המנשק לעבודה עם סביבה הוא אותו מנשק כפי שהוגדר עבור שפת "ויהי".
- במסגרת הפתרון, ניתן לכתוב פונקציות עזר במקרה הצורך.

הפרוצדורה *value-of* (כפי שמופיעה בספר בעמוד 71):

```
;;value-of :  Exp x Env ----> ExpVal
 (define value-of
  (lambda (exp env)
   (cases expression exp
    (const-exp (num) (num-val num))
    (var-exp (var) (apply-env env var))
    (diff-exp (exp1 exp2)
       (let ((val1 (value-of exp1 env))
             (val2 (value-of exp2 env)))
         (let ((num1 (expval->num val1))
               (num2 (expval->num val2)))
          (num-val (- num1 num2)))))
    (zero?-exp (exp1)
      (let ((val1 (value-of exp1 env)))
        (let ((num1 (expval->num val1)))
          (if (zero? num1) (bool-val #t)  (bool-val #f)))))
    (if-exp (exp1 exp2 exp3)
      (let ((val1 (value-of exp1 env)))
        (if (expval->bool val1) (value-of exp2 env) (value-of exp3 env))))
    (let-exp (var exp1 body)
      (let ((val1 (value-of exp1 env)))
         (value-of body (extend-env var val1 env))))
    )))
```

שאלה 2 (25 נקודות)

שאלה זו עוסקת בשדרוג של שפת "וישגור" (שפת PROC) המתוארת בספר הלימוד בפרק 3 בעמודים 74-81. שפה זו מאפשרת הגדרה והפעלה של פרוצדורות לא רקורסיביות.

ברצוננו להרחיב את השפה עם ביטוי חדש שתחבירו נתון על-ידי הדקדוק הבא:

Expression ::= **select [{** ***Expression*** **}$^+$] where** ***Expression***

```
select-exp (exps pred)
```

כאשר,

- exps רשימת ביטויים שערכן מספרי.
- pred ביטוי המייצג פרוצדורה המקבלת מספר כפרמטר ומחזירה ערך בוליאני.

ביטוי select-exp מפעיל את pred על כל אחד מהערכים ברשימה, ומחזיר רשימה המכילה מספרי אינדקס של הערכים מתוך exps שעבורן pred החזיר true. (האינדקסים ימוספרו החל מ-1)

להלן דוגמה להמחשת השימוש בביטוי החדש:

select [10 5 -(12, 2) 70] where proc (x) zero? (-(x,10))

בדוגמה זו, תוחזר כתשובה רשימה המכילה את הערכים 1 ו-3 מאחר ורק באינדקסים 1 ו-3 היו ביטויים שעבורן הוחזר true.

שימו לב: בשאלה זו יש טיפוס נתונים חדש (רשימה) לכן, יש להסביר את כל השינויים שנדרש לבצע בחלקים השונים של המפרש של שפת PROC (שפת "וישגור")

הסבירו ורשמו במחברת הבחינה את כל השינויים הנדרשים במפרש על מנת שיתמוך בביטוי החדש.

שאלה 3 (25 נקודות)

בספר הלימוד בעמודים 113-120 מתוארת שפת **"וירמוז"**(IMPLICIT-REFS).

ברצוננו להרחיב שפה זו עם ביטוי חדש בשם seq.

להלן הדקדוק המגדיר את הביטוי החדש שברצוננו להוסיף:

Expression* ::=** seq (identifier [{ ***Expression}$^{+(,)}$])

seq-exp (id exps)

לביטוי **seq-exp** המרכיבים הבאים:

- **id** – משתנה שכבר הוגדר קודם לכן. (זו תהיה שגיאה לבצע ביטוי זה על משתנה לא קיים)
- **exps** – רשימת ביטויים (לפחות אחד) המופרדים בפסיק.

שלבי חישוב ביטוי seq-exp הם:

למשתנה המצויין ע"י id יבוצעו מספר השמות בצורה סדרתית. ההשמות הם ערכי הביטויים הרשומים ברשימה exps. בתחילה המשתנה יקבל את ערכו של הביטוי הראשון, לאחר מכן את ערכו של הביטוי השני, וכן הלאה. בכל חישוב ביטוי, משתמשים בערכו העדכני של המשתנה id עד כה.
ביטוי זה מחזיר את ערכו של האחרון של המשתנה המצויין ע"י id.

להלן דוגמא:

```
let z= 10
in let y= seq (z [-(z,1), zero? (z), if z then 3 else 6])
   in  y
```

בדוגמה זו, מוגדר משתנה z שערכו 10.

לאחר מכן, מוגדר משתנה y המקבל את ערכו של ביטוי seq.

ביטוי ה-seq משנה בתחילה את ערכו של z להיות 9 (ערכו של הביטוי הראשון), לאחר מכן ערכו של z משתנה לערכו של הביטוי השני, שהוא ביטוי בוליאני, ולכן z משנה את ערכו ל-false ולבסוף, המשתנה z מקבל את ערכו של הביטוי האחרון, במקרה זה 6.

ולכן המשתנה y מקבל ערך של (num-val 6) בהחסן (store) וזה גם ערכה של כל הדוגמה.

(המשך השאלה בעמוד הבא)

א. (7 נקודות)

להלן נתונים ה-**scanner** וה-**parser** של שפת "**וירמוז**". עליכם להשלים את החסר בריבוע המסומן.**<u>את התשובה כיתבו במחברת הבחינה</u>**.

ה-scanner:

```
(define the-lexical-spec
'((whitespace (whitespace) skip)
(comment ("%" (arbno (not #\newline))) skip)
  (identifier(letter (arbno (or letter digit "_" "-" "?")))
           symbol)
  (number (digit (arbno digit)) number)
(number ("-" digit (arbno digit)) number)
))
```

ה-parser:

```
(define the-grammar
'((program (expression) a-program)
  (expression (number) const-exp)
  (expression ("-" "(" expression "," expression ")")
    diff-exp)
  (expression ("zero?" "(" expression ")") zero?-exp)
  (expression
   ("if" expression "then" expression "else" expression)
     if-exp)
  (expression (identifier) var-exp)
  (expression ("let" identifier "=" expression "in" expression)
     let-exp)
  (expression ("proc" "(" identifier ")" expression)
     proc-exp)
  (expression ("(" expression expression ")") call-exp)
  (expression
        ("letrec"
          (arbno identifier "(" identifier ")" "=" expression)
           "in" expression)
letrec-exp)
(expression ("set" identifier "=" expression)
    assign-exp)
(expression (




)
  seq-exp)))
```

את התשובה כיתבו במחברת הבחינה

ב. (18 נקודות)

להלן הפרוצדורות *value-of* ו-*apply-procedure* של שפת **"וירמוז".**

הסבירו במדויק מה הם השינויים והתוספות הנדרשים לשילוב הביטוי החדש בשפה זו.

<u>את התשובה כיתבו במחברת הבחינה</u>.

הערות:

- המנשק לעבודה עם סביבה הוא אותו מנשק כפי שהוגדר עבור שפת "וירמוז".
- המנשק לעבודה עם החסן (store) הוא אותו מנשק כפי שהוגדר עבור שפת "וירמוז".
- במסגרת הפתרון, ניתן לכתוב פונקציות עזר במקרה הצורך.

```
;; value-of : Exp * Env ->ExpVal
;; Page: 118, 119
  (define value-of
    (lambda (exp env)
      (cases expression exp
        (const-exp (num) (num-val num))

       (var-exp (var) (deref (apply-env env var)))

        (diff-exp (exp1 exp2)
          (let ((val1 (value-of exp1 env))
                (val2 (value-of exp2 env)))
            (let ((num1 (expval->num val1))
                  (num2 (expval->num val2)))
              (num-val
                (- num1 num2)))))

        (zero?-exp (exp1)
          (let ((val1 (value-of exp1 env)))
            (let ((num1 (expval->num val1)))
              (if (zero? num1)
                (bool-val #t)
                (bool-val #f)))))

        (if-exp (exp1 exp2 exp3)
          (let ((val1 (value-of exp1 env)))
            (if (expval->bool val1)
              (value-of exp2 env)
              (value-of exp3 env))))

        (let-exp (var exp1 body)
          (let ((v1 (value-of exp1 env)))
            (value-of body
              (extend-env var (newref v1) env))))

        (proc-exp (var body)
          (proc-val (procedure var body env)))

        (call-exp (rator rand)
          (let ((proc (expval->proc (value-of rator env)))
```

```
                (arg (value-of rand env)))
            (apply-procedure proc arg)))


      (letrec-exp (p-names b-vars p-bodies letrec-body)
          (value-of letrec-body
            (extend-env-rec* p-names b-vars p-bodies env)))

         (assign-exp (var exp1)
           (begin
             (setref!
               (apply-env env var)
               (value-of exp1 env))
             (num-val 27))))))


(define apply-procedure
      (lambda (proc1 val)
         (cases proc proc1
            (procedure (var body saved-env)
                (value-of body
                   (extend-env var (newref val) saved-env))))))
```

שאלה 4 (25 נקודות)

להלן נתונה תוכנית בשפת "ויקיש" (INFERRED):

```
((proc (x:?) (x 7)    proc (z:?)  proc (y:?)  -(y,z))  3)
```

ברצוננו לקבוע מהו טיפוס התוכנית (אם קיים).לשם כך, עליכם להשתמש באלגוריתם שנלמד בפרק 7 להקשת הטיפוס.

א. (10 נקודות)

הקצו לביטוי הנתון בשאלה ולתתי הביטויים שלו משתני-טיפוס (Type Variables) מתאימים, וחברו משוואות מתאימות לביטוי הנתון ולתתי הביטויים שלו.

ב. (15 נקודות)

פתרו את המשוואות שהרכבתם בסעיף א' תוך שימוש ב-substitution ו-unification בדומה למתואר בספר בעמודים 252-258. בסיום פתרון המשוואות רשמו מהו הטיפוס שאותו הקשתם עבור התוכנית הנתונה.

ב ה צ ל ח ה !